JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ZoneRulesProvider

java.lang.Object
    java.time.zone.ZoneRulesProvider

```
public abstract class ZoneRulesProvider
extends Object
```

Provider of time-zone rules to the system.

This class manages the configuration of time-zone rules. The static methods provide the public API that can be used to manage the providers. The abstract methods provide the SPI that allows rules to be provided.

ZoneRulesProvider may be installed in an instance of the Java Platform as extension classes, that is, jar files placed into any of the usual extension directories. Installed providers are loaded using the service-provider loading facility defined by the `ServiceLoader` class. A ZoneRulesProvider identifies itself with a provider configuration file named `java.time.zone.ZoneRulesProvider` in the resource directory `META-INF/services`. The file should contain a line that specifies the fully qualified concrete zonerules-provider class name. Providers may also be made available by adding them to the class path or by registering themselves via `registerProvider(ZoneRulesProvider)` method.

The Java virtual machine has a default provider that provides zone rules for the time-zones defined by IANA Time Zone Database (TZDB). If the system property `java.time.zone.DefaultZoneRulesProvider` is defined then it is taken to be the fully-qualified name of a concrete ZoneRulesProvider class to be loaded as the default provider, using the system class loader. If this system property is not defined, a system-default provider will be loaded to serve as the default provider.

Rules are looked up primarily by zone ID, as used by `ZoneId`. Only zone region IDs may be used, zone offset IDs are not used here.

Time-zone rules are political, thus the data can change at any time. Each provider will provide the latest rules for each zone ID, but they may also provide the history of how the rules changed.

Implementation Requirements:

This interface is a service provider that can be called by multiple threads. Implementations must be immutable and thread-safe.

Providers must ensure that once a rule has been seen by the application, the rule must continue to be available.

Providers are encouraged to implement a meaningful `toString` method.

Many systems would like to update time-zone rules dynamically without stopping the JVM. When examined in detail, this is a complex problem. Providers may choose to handle dynamic updates, however the default provider does not.

Since:

1.8

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	ZoneRulesProvider()	Constructor.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
static Set<String>	getAvailableZoneIds()	Gets the set of available zone IDs.
static ZoneRules	getRules(String zoneId, boolean forCaching)	Gets the rules for the zone ID.
static NavigableMap<String, ZoneRules>	getVersions (String zoneId)	Gets the history of rules for the zone ID.
protected boolean	provideRefresh()	SPI method to refresh the rules from the underlying data provider.
protected abstract ZoneRules	provideRules (String zoneId, boolean forCaching)	SPI method to get the rules for the zone ID.
protected abstract NavigableMap<String, ZoneRules>	provideVersions (String zoneId)	SPI method to get the history of rules for the zone ID.
protected abstract Set<String>	provideZoneIds()	SPI method to get the available zone IDs.
static boolean	refresh()	

		Refreshes the rules from the underlying data provider.
static void	**registerProvider**(ZoneRulesProvider provi	Registers a zone rules provider.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

ZoneRulesProvider

```
protected ZoneRulesProvider()
```

Constructor.

Method Details

getAvailableZoneIds

```
public static Set<String> getAvailableZoneIds()
```

Gets the set of available zone IDs.

These IDs are the string form of a ZoneId.

Returns:

the unmodifiable set of zone IDs, not null

getRules

```
public static ZoneRules getRules(String zoneId,
                                 boolean forCaching)
```

Gets the rules for the zone ID.

This returns the latest available rules for the zone ID.

This method relies on time-zone data provider files that are configured. These are loaded using a ServiceLoader.

The caching flag is designed to allow provider implementations to prevent the rules being cached in ZoneId. Under normal circumstances, the caching of

zone rules is highly desirable as it will provide greater performance. However, there is a use case where the caching would not be desirable, see `provideRules(String, boolean)`.

Parameters:

`zoneId` - the zone ID as defined by `ZoneId`, not null

`forCaching` - whether the rules are being queried for caching, true if the returned rules will be cached by `ZoneId`, false if they will be returned to the user without being cached in `ZoneId`

Returns:

the rules, null if `forCaching` is true and this is a dynamic provider that wants to prevent caching in `ZoneId`, otherwise not null

Throws:

`ZoneRulesException` - if rules cannot be obtained for the zone ID

getVersions

```
public static NavigableMap<String, ZoneRules> getVersions
(String zoneId)
```

Gets the history of rules for the zone ID.

Time-zones are defined by governments and change frequently. This method allows applications to find the history of changes to the rules for a single zone ID. The map is keyed by a string, which is the version string associated with the rules.

The exact meaning and format of the version is provider specific. The version must follow lexicographical order, thus the returned map will be order from the oldest known rules to the newest available rules. The default 'TZDB' group uses version numbering consisting of the year followed by a letter, such as '2009e' or '2012f'.

Implementations must provide a result for each valid zone ID, however they do not have to provide a history of rules. Thus the map will always contain one element, and will only contain more than one element if historical rule information is available.

Parameters:

`zoneId` - the zone ID as defined by `ZoneId`, not null

Returns:

a modifiable copy of the history of the rules for the ID, sorted from oldest to newest, not null

Throws:

ZoneRulesException - if history cannot be obtained for the zone ID

registerProvider

```
public static void registerProvider(ZoneRulesProvider provider)
```

Registers a zone rules provider.

This adds a new provider to those currently available. A provider supplies rules for one or more zone IDs. A provider cannot be registered if it supplies a zone ID that has already been registered. See the notes on time-zone IDs in ZoneId, especially the section on using the concept of a "group" to make IDs unique.

To ensure the integrity of time-zones already created, there is no way to deregister providers.

Parameters:

provider - the provider to register, not null

Throws:

ZoneRulesException - if a zone ID is already registered

refresh

```
public static boolean refresh()
```

Refreshes the rules from the underlying data provider.

This method allows an application to request that the providers check for any updates to the provided rules. After calling this method, the offset stored in any ZonedDateTime may be invalid for the zone ID.

Dynamic update of rules is a complex problem and most applications should not use this method or dynamic rules. To achieve dynamic rules, a provider implementation will have to be written as per the specification of this class. In addition, instances of ZoneRules must not be cached in the application as they will become stale. However, the boolean flag on provideRules(String, boolean) allows provider implementations to control the caching of ZoneId, potentially ensuring that all objects in the system see the new rules. Note that there is likely to be a cost in performance of a dynamic rules provider. Note also that no dynamic rules provider is in this specification.

Returns:

true if the rules were updated

Throws:

ZoneRulesException - if an error occurs during the refresh

provideZoneIds

```
protected abstract Set<String> provideZoneIds()
```

SPI method to get the available zone IDs.

This obtains the IDs that this `ZoneRulesProvider` provides. A provider should provide data for at least one zone ID.

The returned zone IDs remain available and valid for the lifetime of the application. A dynamic provider may increase the set of IDs as more data becomes available.

Returns:

the set of zone IDs being provided, not null

Throws:

`ZoneRulesException` - if a problem occurs while providing the IDs

provideRules

```
protected abstract ZoneRules provideRules(String zoneId,
                                          boolean forCaching)
```

SPI method to get the rules for the zone ID.

This loads the rules for the specified zone ID. The provider implementation must validate that the zone ID is valid and available, throwing a `ZoneRulesException` if it is not. The result of the method in the valid case depends on the caching flag.

If the provider implementation is not dynamic, then the result of the method must be the non-null set of rules selected by the ID.

If the provider implementation is dynamic, then the flag gives the option of preventing the returned rules from being cached in `ZoneId`. When the flag is true, the provider is permitted to return null, where null will prevent the rules from being cached in `ZoneId`. When the flag is false, the provider must return non-null rules.

Parameters:

`zoneId` - the zone ID as defined by `ZoneId`, not null

`forCaching` - whether the rules are being queried for caching, true if the returned rules will be cached by `ZoneId`, false if they will be returned to the user without being cached in `ZoneId`

Returns:

the rules, null if `forCaching` is true and this is a dynamic provider that wants to prevent caching in `ZoneId`, otherwise not null

Throws:

`ZoneRulesException` - if rules cannot be obtained for the zone ID

provideVersions

```
protected abstract NavigableMap<String, ZoneRules> provideVersions
(String zoneId)
```

SPI method to get the history of rules for the zone ID.

This returns a map of historical rules keyed by a version string. The exact meaning and format of the version is provider specific. The version must follow lexicographical order, thus the returned map will be order from the oldest known rules to the newest available rules. The default 'TZDB' group uses version numbering consisting of the year followed by a letter, such as '2009e' or '2012f'.

Implementations must provide a result for each valid zone ID, however they do not have to provide a history of rules. Thus the map will contain at least one element, and will only contain more than one element if historical rule information is available.

The returned versions remain available and valid for the lifetime of the application. A dynamic provider may increase the set of versions as more data becomes available.

Parameters:

`zoneId` - the zone ID as defined by `ZoneId`, not null

Returns:

a modifiable copy of the history of the rules for the ID, sorted from oldest to newest, not null

Throws:

`ZoneRulesException` - if history cannot be obtained for the zone ID

provideRefresh

```
protected boolean provideRefresh()
```

SPI method to refresh the rules from the underlying data provider.

This method provides the opportunity for a provider to dynamically recheck the underlying data provider to find the latest rules. This could be used to load new rules without stopping the JVM. Dynamic behavior is entirely optional and most providers do not support it.

This implementation returns false.

Returns:

true if the rules were updated

Throws:

`ZoneRulesException` - if an error occurs during the refresh

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.